

FLA_VALIDATION

vetted growing-memory 구현 축별 검증 (정확 구현 먼저)

"정확 구현 먼저 → autoresearch 연결" 원칙으로, flash-linear-attention(fla) + titans-pytorch의 검증된 모듈을 단일 고정 LayerNorm 스캐폴드에 끼워 같은 MQAR로 검증. 재현: packages/growing-memory/fla_validate.py .

결과 (MQAR vocab32/pairs2/L64, dim128, 4k step, 4090, chance≈0.062)

구현	recall	grok	시간	base_rule 매핑
deltanet (fla)	1.0	즉시	41s	delta rule
gated_deltanet (fla)	1.0	즉시	120s	Titans류(게이트+델타)
retention (fla)	1.0	@3000	29s	retention(스칼라 감쇠)
titans (lucidrains)	1.0	@3000	113s	Titans 논문
linear (fla)	0.54	부분	23s	순수 선형(회상 약점)
gla/dla (fla)	0.06	chance	21s	GLA(게이트) — 추가 튜닝 필요

해석

- 4/6 vetted 구현이 정확회상 1.0 — 빠르고 완벽(특히 deltanet: O(1) 상태 + 1.0 + 41초 최고).
- 내 from-scratch 근사(model.py의 linear/dla/titans)는 부분해/느린 grok였으나, vetted 구현은 즉시 1.0.
- 순수 선형(fla linear)은 0.54로 회상 약점 재확인(이론과 일치). GLA는 chance — 게이트 초기화/스텝 추가 필요 (별도 튜닝 대상).
- 검증된 정답 셋: deltanet, gated_deltanet, retention, titans → 이들을 autoresearch base_rule로 연결.

autoresearch 연결 (검증된 구현을 base_rule로)

검증 통과 구현을 real/model.py 의 MemoryMixer에 base_rule 로 연결(ext_mem):
deltanet/gated_deltanet/retention/gla/linear_fla (fla) + titans_real (lucidrains). 스캐폴드는 검증과 동일하게 LayerNorm으로 정렬(기존 RMSNorm은 grok이 늦었음). fla 커널은 bf16 autocast.

연결 후 autoresearch 하니스(GrowingMemoryModel+chunked, seq64/pairs2/4k) 실행: | base_rule | recall | 비교 |
| ---|---|---| | deltanet | 1.0 (grok@1000, hard 0.97) | end-to-end 연결 검증  || retention | 0.55 | 클린 스캐폴드는 1.0이나 풀 하니스는 4k 내 grok 안 됨(late-grokker, 민감) |

→ deltanet으로 "정확 구현 → autoresearch 연결" 입증(1.0 end-to-end). retention은 클린 스캐폴드 검증 1.0이나 풀 하니스 정렬은 추가 폴리시 필요(weight tying/SSL front/스텝). 스위치에는 deltanet 등 검증·연결된 base_rule을 사

용. 재현: `grok_tune.py --variants deltanet,retention,titans_real,...`.

교훈 적용

- 동일 고정 스캐폴드(LayerNorm) + 같은 시드/평가셋 → 공정 비교(이전 RMSNorm 스캐폴드 혼재 문제 제거).
- from-scratch보다 **vetted 라이브러리 채택**이 정확·빠름(이번 세션 핵심 교훈).